Implementation prompt: spawn and preview Wall-E in MuJoCo

Use this prompt with a coding agent in /home/varun/Intelligent_systems/Wall-E/wall-e-rl. Implement the preview, verify it, and actually open the viewer when a graphical desktop is available. Work incrementally and finish the preview milestone before progressing to robot dynamics or RL.

1. Objective and boundaries

Bring the existing Wall-E CAD assembly into MuJoCo with its complete appearance, correct scale, preserved component placements, and correct wheel articulation. Create a reproducible local conversion and preview workflow. Preserve the original URDF and meshes. No new Onshape export is required to address the identified mass problems.

This task authorizes implementation, necessary project-local dependencies, verification, screenshots, and opening a local viewer. It does not authorize PPO training, hardware firmware, motor actuation, or inventing measured parameters. A static or paused geometry preview is an acceptable first milestone. Do not describe it as a physically validated robot simulator. Missing dynamics parameters must not prevent a geometry-only preview.

Read AGENTS.md, docs/SESSION_CONTEXT.md, docs/HARDWARE.md, docs/PLAN.md, docs/PROJECT_MAP.md, configs/robot.yaml, and the two MuJoCo workflow PDFs. Preserve unrelated user edits. Keep simulation, training, and deployment separate, with independent safety logic outside any eventual policy.

2. Resolve the input and runtime

The most recently inspected source is Assembly-Wall-E/assembly_1/urdf/assembly_1.urdf with meshes under Assembly-Wall-E/assembly_1/meshes/. Verify that these paths still exist and select the intended weighted export; do not silently switch to the old Wall-E-Assembly export. If names changed, inventory candidate exports and report the selected path.

System python3 currently imports MuJoCo 3.7.0 and mujoco.viewer. The project .venv was last checked without MuJoCo. Recheck both interpreters. Prefer the project environment for the completed workflow; install mujoco==3.7.0 there if needed and record the version as an optional simulation dependency in pyproject.toml. Install only necessary mesh-processing dependencies, record their versions, and do not upgrade unrelated RL packages. System Python remains a usable initial diagnostic runtime.

Run a tiny headless MuJoCo compilation/forward-pass check before handling the robot. Check DISPLAY/WAYLAND_DISPLAY, GLFW/OpenGL availability, and whether a graphical desktop is available. Module import alone does not prove GUI rendering works. Never request account passwords or API keys; the local export is sufficient for this task.

3. Audit the export before conversion

Parse the URDF as XML. Validate unique link/joint names, one connected rooted tree, valid parent/child references, finite transforms and numeric values, and existing mesh paths. Preserve namespaces if present. Save input hashes, interpreter and dependency versions, mesh counts, missing files, joint types, and source mass totals to an audit JSON artifact.

Previous inspection found 553 links, 550 fixed joints, two continuous wheel joints, and no collision elements. Fixed-joint connectivity gives three rigid groups: chassis and two wheels. The source mass total was approximately 238.833 g, which is not a measured assembled mass. Recompute these values instead of hard-coding them.

Raw MuJoCo loading failed with 'inertia must have positive eigenvalues' at 0603_resistor_004_2. Retaining visual geometry and enabling fixed-body fusion still failed. Do not rely on fusestatic alone to repair invalid

input tensors. Do not patch every resistor with arbitrary tiny mass/inertia or hide bad properties using global boundmass, boundinertia, balanceinertia, or settotalmass settings.

4. Preserve geometry, coordinates, and wheel motion

Resolve package://assembly_1/... references against the local exported package. Use portable relative paths or a deliberate asset resolver rather than depending on ROS package lookup. Register unique mesh assets and retain repeated geometry instances at distinct transforms. Reusing one mesh file for several instances is valid and must not eliminate their placements.

Compute transforms along the complete URDF tree at zero joint configuration: parent transform multiplied by joint-origin transform, then visual-origin transform. URDF rpy uses radians; use the correct rotation convention. Apply mesh scale exactly once. MuJoCo quaternion order is w, x, y, z. Transform hinge axes in the correct joint frame, not just the original global coordinates. Preserve both wheel signs; do not infer left/right solely from generic names.

Compute rigid groups by fixed-joint connectivity. Keep a chassis group and the two wheel groups, with rims and tires rigidly attached within each wheel. Build a source-link-to-output-body mapping. Keep the visual geoms in the appropriate bodies while removing the hundreds of unnecessary physical link inertias.

Use a documented frame such as X forward, Y left, Z up. If the export uses another frame, transform the whole robot consistently, including mesh origins, center of mass, inertias, and joint axes. A robot origin at axle midpoint is useful, but do not change frames by eye or recenter each mesh independently. MuJoCo may internally recenter meshes; avoid applying that offset a second time.

5. Correct mass and inertia without double counting

User-confirmed inputs: complete wheel 0.040 kg each; complete motor 0.029 kg each; screw 0.0045 kg each; populated-board target 0.030 kg, subject to confirming exactly which parts were weighed; three batteries plus their holder total 0.153 kg. The user intentionally used one effective density across the battery-and-holder group. Preserve that shared-density assumption rather than automatically forcing a separate 0.135 kg battery mass and 0.018 kg holder mass distribution.

The export contains all three cells in Battery18650__NONE_2.stl, but the associated inertia and mass correspond to one cell. BatteryCaseSpring__Cut_Extrude2.stl contains six springs while its inertial data describe one spring. Other instance links have negligible placeholder masses. The inspected numbers satisfy $3 \times 38.9855\ g + 35.2461\ g + 6 \times 0.132943\ g = 153.000258\ g$. Treat this as evidence of lost instance properties; re-audit the current export before using it. The export mechanism itself has not been conclusively established.

The four screw_m3_75mm instances previously exported only 0.000603186 g each despite the user reporting 4.5 g in Onshape. Correct each actual screw instance to the confirmed measurement. The wheel groups already totaled 40 g each, and the complete motors including shafts totaled approximately 29 g each. Apply measured mass once per weighed assembly, not separately to every solid inside it.

Audit repeated PCB/header/electronics parts for the same geometry-versus-inertia mismatch. Do not automatically trust the rest of the source inertias merely because the battery discrepancy is understood. Define explicit membership for each weighed group and check for overlapping membership. If the exact 30 g board membership or any additional part mass is ambiguous, record it as unresolved instead of assigning unexplained mass.

For reliable solid meshes, calculate volume, volume centroid, and inertia under the declared uniform-density assumption. Account for disconnected solids, vertex welding tolerance, watertightness, consistent winding, and unit conversion. Integrate complete geometry, not one representative instance. Do not fill every cavity or take a convex hull and silently call that the measured component. Intersecting source solids require a documented choice about additive versus union volume. For the battery/holder assumption, match the intended selected-solid distribution and document the method.

Set effective density to target group mass divided by total modeled solid volume. Correct both center of mass and inertia, not just the URDF mass scalar. Rotate tensors into a common body frame and aggregate

about the combined center of mass using I = sum(R_i I_i R_i^T + m_i[(d_i dot d_i)E - d_i d_i^T]). Confirm tensor symmetry, positive principal inertias, physical triangle inequalities, and finite values. Retain sufficient numeric precision.

The listed components with three batteries and four screws sum to 0.339 kg before unlisted parts. This is a bookkeeping check, not permission to force the model to 0.339 kg or call it the complete measured robot mass. Do not scale everything globally to hide discrepancies. Record measured targets, reconstructed estimates, assumptions, unresolved masses, and corrected source-to-output values in a mass audit.

If reliable physical inertias cannot yet be reconstructed, complete the static visual preview first. It may omit dynamic inertial blocks entirely because there are no moving degrees of freedom. Mark it geometry-only and keep it separate from any articulated model. A physical dynamic model remains gated on the missing characterization data.

6. Produce native MuJoCo assets and scenes

Create a reproducible conversion script under simulation/, using a package __init__.py if module execution is provided. Suggested outputs are simulation/models/wall_e_preview.xml and, only when inertias are defensible, an articulated inspection model with two hinges. Do not create a supposedly calibrated motor-driven model to satisfy this preview task.

Keep the detailed electronics as visual geoms with contype=0 and conaffinity=0. Retain URDF RGBA values if present. If colors are missing, use a clearly documented cosmetic palette; do not imply recovery of original CAD materials. Supply explicit aggregate inertias for articulated bodies and disable unintended mass inference from visual details. There must be no double counting of explicit body mass and visual geometry mass.

MuJoCo can reject degenerate or nearly planar meshes even when used visually. Identify and report affected meshes. Prefer documented visual-only replacements or conservative simplification where necessary, preserving major hardware geometry. Never silently omit wheels, batteries, chassis, or the PCB. If tiny decorative geometry is omitted, list exactly what and why. Validate any mesh processing against assembly dimensions and poses.

For the preview, add a neutral floor/grid, lighting, suitable background, and a camera centered on robot bounds with a distance proportional to its size. Place the robot so it is fully visible and does not visually intersect the floor. The preview floor may be visual-only. Do not infer realistic tire friction, motor torque, gear losses, latency, or safety thresholds from appearance.

Use a static model or a paused articulated model for the first render so the robot does not fall away immediately. A later dynamic scene will need a free chassis joint, floor contact, simple collision shapes, and characterized actuation; those additions are not required for the geometry preview. Label all preview-only settings.

7. Implement the viewer and screenshot workflow

Provide a documented launch command from the project root, preferably .venv/bin/python -m simulation.preview --model simulation/models/wall_e_preview.xml. Make the model argument optional with a sensible default, resolve paths independently of the caller’s current directory, and print the actual interpreter, MuJoCo version, model path, and preview mode.

Use mujoco.MjModel.from_xml_path, create MjData, call mj_forward, and open mujoco.viewer.launch_passive. Set camera lookat/distance/azimuth/elevation while holding the viewer lock, then sync. Keep the preview frozen by default: do not call mj_step in an uncalibrated visual preview. Explain that wheel animation, if offered, is a kinematic inspection and not motor simulation. Keep a modest-rate event loop until the user closes the window; do not immediately exit after launching.

Support a screenshot mode with MuJoCo Renderer and a saved PNG path. Choose a supported headless backend such as EGL or OSMesa only after checking availability, and set MUJOCO_GL before importing MuJoCo when necessary. Interactive GUI rendering and offscreen rendering may require different processes/backends. Do not claim an offscreen screenshot proves the interactive window opened.

Actually launch the GUI when a desktop/display is available. Inspect robot visibility and framing; leave the preview open for the user. If GUI access is unavailable, retain the compiled model, attempt offscreen rendering, provide the screenshot and exact local launch command, and state the concrete display/backend limitation. Do not install system graphics packages or alter drivers without specific need and authority.

8. Verification and acceptance criteria

Verify transformed mesh placements against the original assembly at zero wheel angles. Report bounding dimensions, wheel centers, axle direction, and whether all three battery cells and four screws are represented. CAD-derived dimensions are estimates to physically cross-check, not newly measured hardware facts.

Compile every delivered model successfully. Check articulated wheel axes and grouping, finite positions, valid aggregate inertias where used, and no missing assets. Save dependency versions, conversion settings, source hashes, and audits. Run targeted numerical checks for transform composition, group membership, mass accounting, and parallel-axis aggregation; avoid broad unrelated tests.

Render and inspect at least a three-quarter overview and another angle that shows the battery group and wheels. Confirm the robot is visible, recognizable, correctly scaled, fully framed, and not an empty frame or clipped geometry. Save screenshots in a preview artifact directory. Verify GUI behavior separately if it is claimed.

The task is complete when a reproducibly generated model compiles and renders, the viewer is open if the desktop permits it, audit artifacts explain all property corrections/omissions, and the user has a one-command way to reopen it. Missing calibration must be explicitly documented; do not proceed to RL training.

9. Documentation and final handoff

Update docs/PROJECT_MAP.md for actual new files, docs/PLAN.md for the preview milestone, docs/HARDWARE.md/configs/robot.yaml only for confirmed values and explicitly labeled CAD estimates, and AGENTS.md for the handoff. Keep complete measured mass and unmeasured motor/IMU parameters unknown. Do not overwrite previous PDFs or claim Phase 3 complete based only on a preview.

In the final response provide clickable links to the MJCF, converter, viewer script, screenshots, and audits; the exact launch and rebuild commands; dependency versions; what was verified; and any remaining limitations. Explain briefly how URDF geometry becomes MuJoCo bodies and why visual preview differs from physical dynamics. Do the authorized work instead of stopping at a plan.

Official references

- MuJoCo Python/viewer: https://mujoco.readthedocs.io/en/latest/python.html
- MuJoCo model and URDF conversion: https://mujoco.readthedocs.io/en/latest/modeling.html#urdf-extensions
- MuJoCo inertial/compiler/mesh reference: https://mujoco.readthedocs.io/en/latest/XMLreference.html